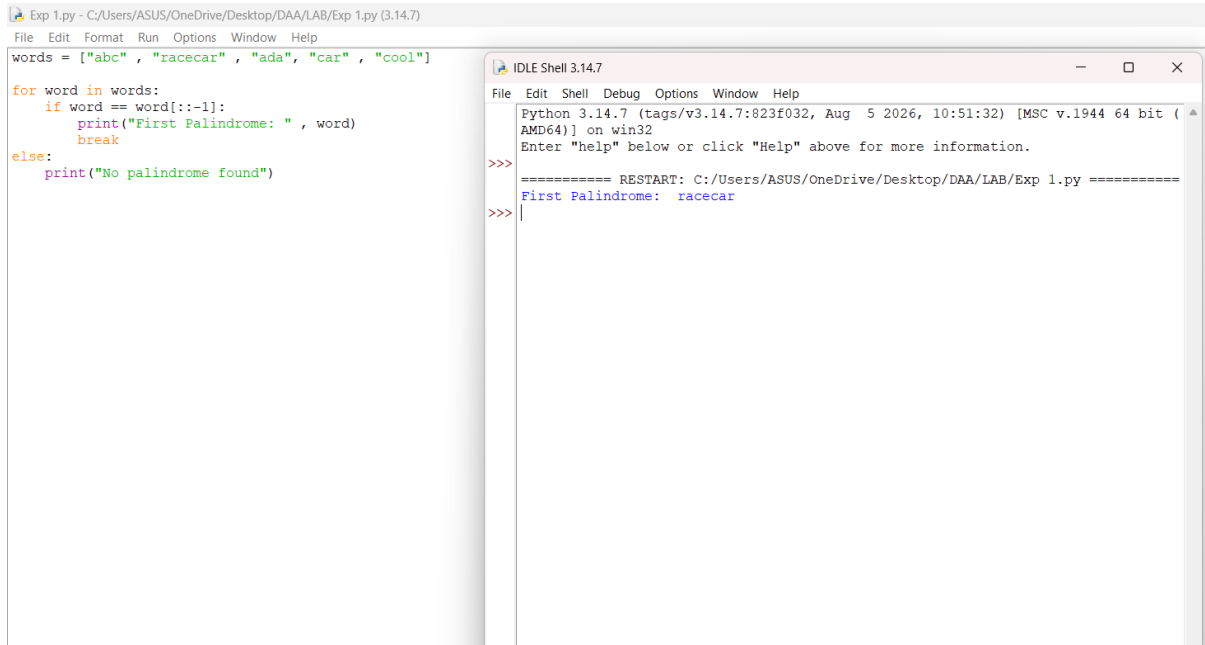


LAB EXPERIMENTS (CHAPTER 1)

1. First Palindromic String



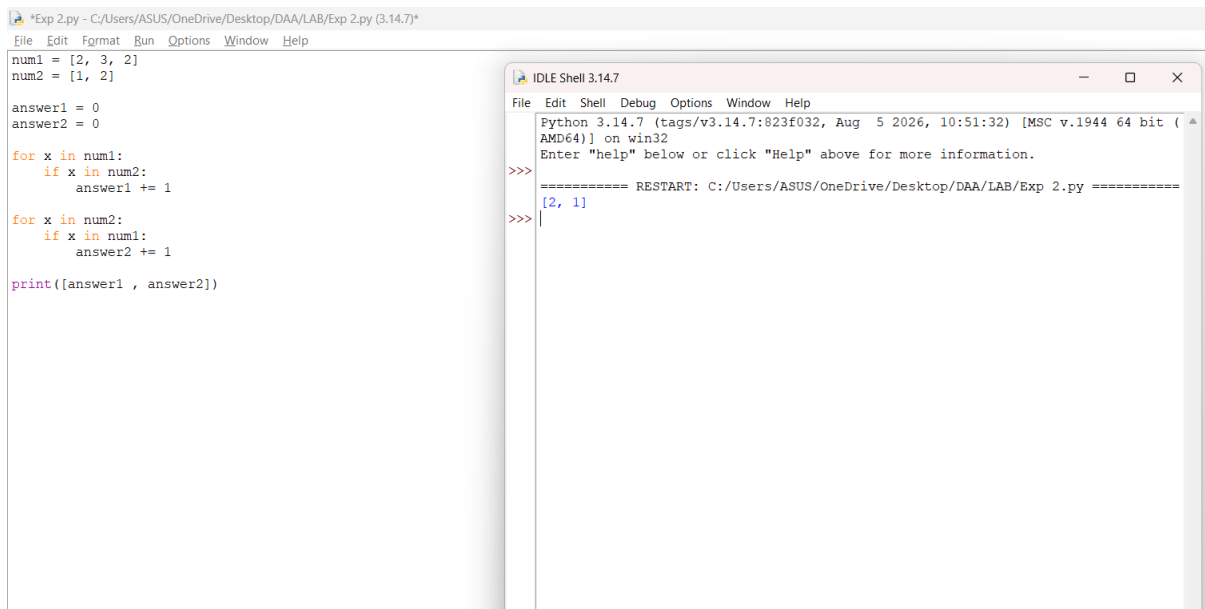
The screenshot shows a Python IDE window titled "Exp 1.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 1.py (3.14.7)". The code defines a list of words: ["abc", "racecar", "ada", "car", "cool"]. It iterates through the list, checking if each word is a palindrome (word == word[::-1]). The first palindrome found is "racecar", which is printed. The IDE Shell window shows the execution output: "First Palindrome: racecar".

```
words = ["abc", "racecar", "ada", "car", "cool"]

for word in words:
    if word == word[::-1]:
        print("First Palindrome: ", word)
        break
else:
    print("No palindrome found")
```

```
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 1.py =====
First Palindrome: racecar
>>>
```

2. Count Common Elements in Two Arrays



The screenshot shows a Python IDE window titled "*Exp 2.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 2.py (3.14.7)*". The code defines two arrays: num1 = [2, 3, 2] and num2 = [1, 2]. It counts the number of common elements by iterating through num1 and checking if each element is in num2. The counts are stored in answer1 and answer2. The final output is printed as a tuple: (answer1, answer2). The IDE Shell window shows the execution output: "[2, 1]".

```
num1 = [2, 3, 2]
num2 = [1, 2]

answer1 = 0
answer2 = 0

for x in num1:
    if x in num2:
        answer1 += 1

for x in num2:
    if x in num1:
        answer2 += 1

print((answer1, answer2))
```

```
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 2.py =====
[2, 1]
>>>
```

3. Sum of Squares of Distinct Counts

The screenshot shows a Python IDE window titled "Exp 3.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 3.py (3.14.7)". The code defines a list `nums = [1, 2, 1]`, initializes `total = 0`, and iterates through the list to count pairs of equal elements. The output is `15`. The IDE Shell window shows the execution output, including the restart command and the result `15`.

```
nums = [1, 2, 1]

total = 0
n = len(nums)

for i in range(n):
    distinct = set()

    for j in range(i, n):
        distinct.add(nums[j])
        count = len(distinct)
        total += count * count

print(total)
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 3.py =====
15
>>>
```

4. Count Equal Pairs Divisible by K

The screenshot shows a Python IDE window titled "Exp 4.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 4.py (3.14.7)". The code defines a list `nums = [3, 1, 2, 2, 2, 1, 3]` and a variable `k = 2`. It iterates through the list to count pairs of equal elements whose sum is divisible by `k`. The output is `4`. The IDE Shell window shows the execution output, including the restart command and the result `4`.

```
nums = [3, 1, 2, 2, 2, 1, 3]
k = 2

count = 0
n = len(nums)

for i in range(n):
    for j in range(i + 1, n):
        if nums[i] == nums[j] and (i * j) % k == 0:
            count += 1

print(count)
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 4.py =====
4
>>>
```

5. Find the Maximum Element in an Array

The screenshot shows a Python IDE window titled "Exp 5.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 5.py (3.14.7)". The code defines a list `nums = [1, 2, 3, 4, 5]` and finds the maximum element. The output is `5`. The IDE Shell window shows the execution output, including the restart command and the result `5`.

```
nums = [1, 2, 3, 4, 5]

maximum = nums[0]

for x in nums:
    if x > maximum:
        maximum = x

print(maximum)
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 5.py =====
5
>>>
```

6. Sort List and Find Maximum

The screenshot shows a Python IDE with a file named 'Exp 6.py'. The code defines a list 'nums' with values [5, 2, 8, 3, 1]. It checks if the list is empty; if not, it sorts the list and prints the maximum value (nums[-1]). The output in the IDLE Shell is 8.

```
Exp 6.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 6.py (3.14.7)
File Edit Format Run Options Window Help
nums = [5, 2, 8, 3, 1]

if len(nums) == 0:
    print("List is empty")
else:
    nums.sort()
    maximum = nums[-1]
    print(maximum)

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 6.py =====
8
>>>
```

7. Find Unique Elements in a List

The screenshot shows a Python IDE with a file named 'Exp 7.py'. The code defines a list 'nums' with values [3, 7, 3, 5, 2, 5, 9, 2]. It creates an empty list 'unique' and iterates through 'nums', adding elements to 'unique' only if they are not already present. The output in the IDLE Shell is [3, 7, 5, 2, 9].

```
Exp 7.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 7.py (3.14.7)
File Edit Format Run Options Window Help
nums = [3, 7, 3, 5, 2, 5, 9, 2]

unique = []

for x in nums:
    if x not in unique:
        unique.append(x)

print(unique)

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 7.py =====
[3, 7, 5, 2, 9]
>>>
```

8. Bubble Sort

The screenshot shows a Python IDE with a file named 'Exp 8.py'. The code defines a list 'nums' with values [5, 3, 8, 1, 2]. It implements the Bubble Sort algorithm by iterating through the list and swapping adjacent elements if they are in the wrong order. The output in the IDLE Shell is [1, 2, 3, 5, 8].

```
Exp 8.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 8.py (3.14.7)
File Edit Format Run Options Window Help
nums = [5, 3, 8, 1, 2]

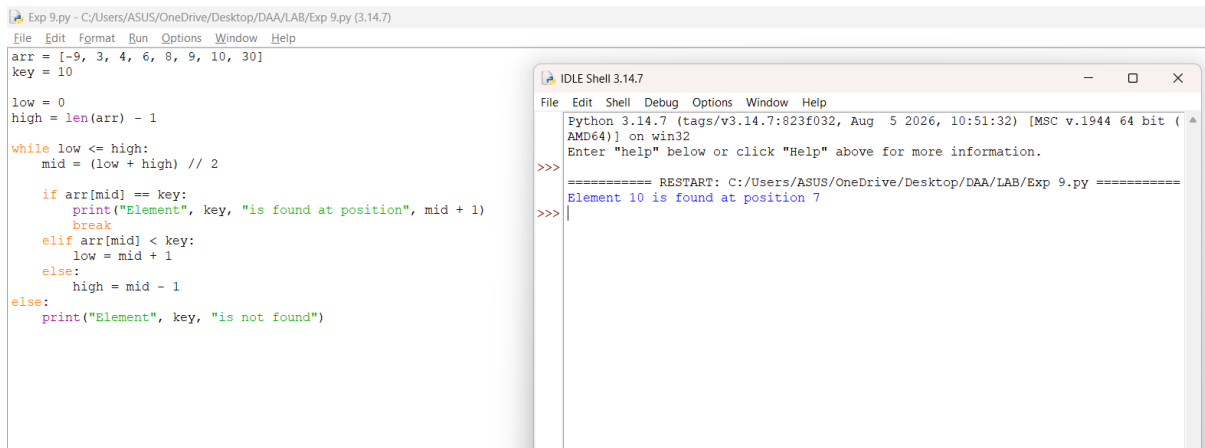
n = len(nums)

for i in range(n):
    for j in range(0, n - i - 1):
        if nums[j] > nums[j + 1]:
            nums[j], nums[j + 1] = nums[j + 1], nums[j]

print(nums)

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 8.py =====
[1, 2, 3, 5, 8]
>>>
```

9. Binary Search



The screenshot shows a Python IDE window titled 'Exp 9.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 9.py (3.14.7)'. The code implements a binary search algorithm on an array. The output window shows the execution result: 'Element 10 is found at position 7'.

```
arr = [-9, 3, 4, 6, 8, 9, 10, 30]
key = 10

low = 0
high = len(arr) - 1

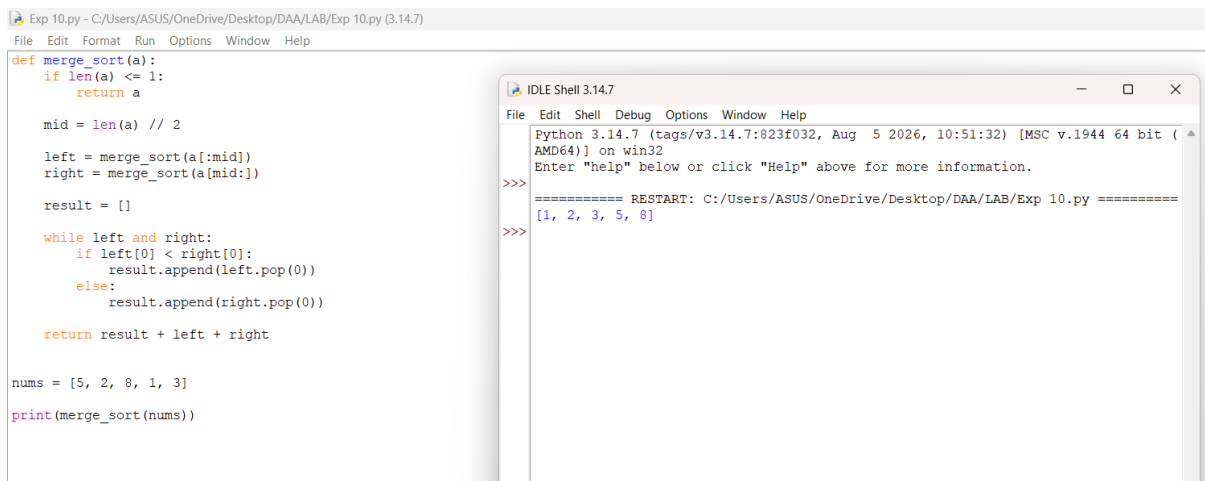
while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element", key, "is found at position", mid + 1)
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
else:
    print("Element", key, "is not found")
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 9.py =====
>>> Element 10 is found at position 7
>>> |
```

10. Sort Array Using Merge Sort



The screenshot shows a Python IDE window titled 'Exp 10.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 10.py (3.14.7)'. The code implements a merge sort algorithm. The output window shows the execution result: '[1, 2, 3, 5, 8]'.

```
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2

    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []

    while left and right:
        if left[0] < right[0]:
            result.append(left.pop(0))
        else:
            result.append(right.pop(0))

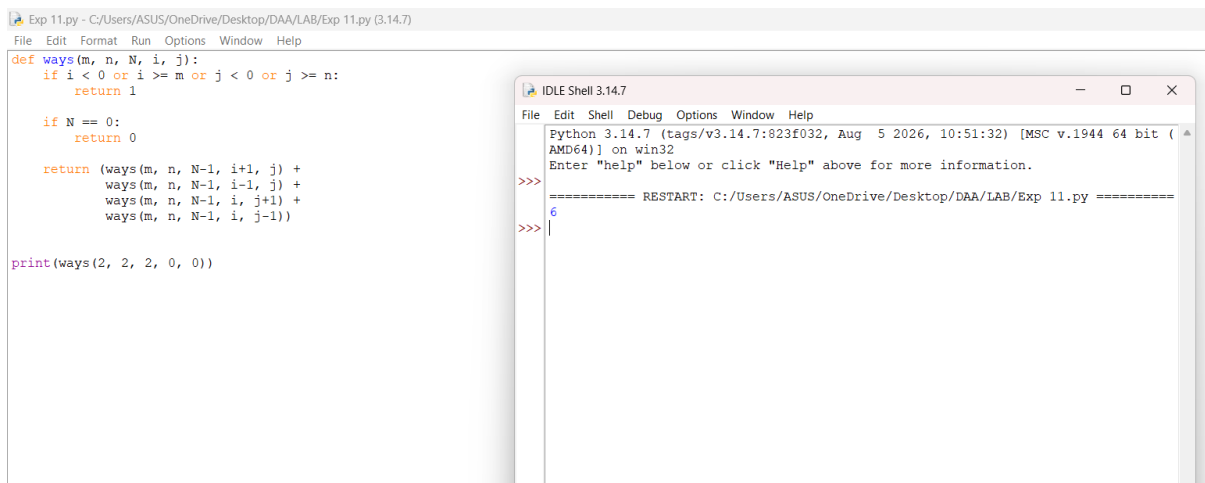
    return result + left + right

nums = [5, 2, 8, 1, 3]
print(merge_sort(nums))
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 10.py =====
>>> [1, 2, 3, 5, 8]
>>> |
```

11. Number of Ways to Move Ball Out of Grid



The screenshot shows a Python IDE window titled 'Exp 11.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 11.py (3.14.7)'. The code implements a recursive function to calculate the number of ways to move a ball out of a grid. The output window shows the execution result: '6'.

```
def ways(m, n, N, i, j):
    if i < 0 or i >= m or j < 0 or j >= n:
        return 1

    if N == 0:
        return 0

    return (ways(m, n, N-1, i+1, j) +
            ways(m, n, N-1, i-1, j) +
            ways(m, n, N-1, i, j+1) +
            ways(m, n, N-1, i, j-1))

print(ways(2, 2, 2, 0, 0))
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 11.py =====
>>> 6
>>> |
```

12. House Robber – Circular Street

```
Exp 12.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 12.py (3.14.7)
File Edit Format Run Options Window Help

def rob(a):
    if len(a) == 1:
        return a[0]

    def simple(a):
        prev = curr = 0
        for money in a:
            prev, curr = curr, max(curr, prev + money)
        return curr

    return max(simple(a[1:]), simple(a[:-1]))

nums = [2, 3, 2]
print(rob(nums))

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 12.py =====
3
>>>
```

13. Climbing Stairs

```
Exp 13.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 13.py (3.14.7)
File Edit Format Run Options Window Help

def climb(n):
    a = 1
    b = 1
    for i in range(n):
        a, b = b, a + b
    return a

print(climb(4))

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 13.py =====
5
>>>
```

14. Unique Paths in a Grid

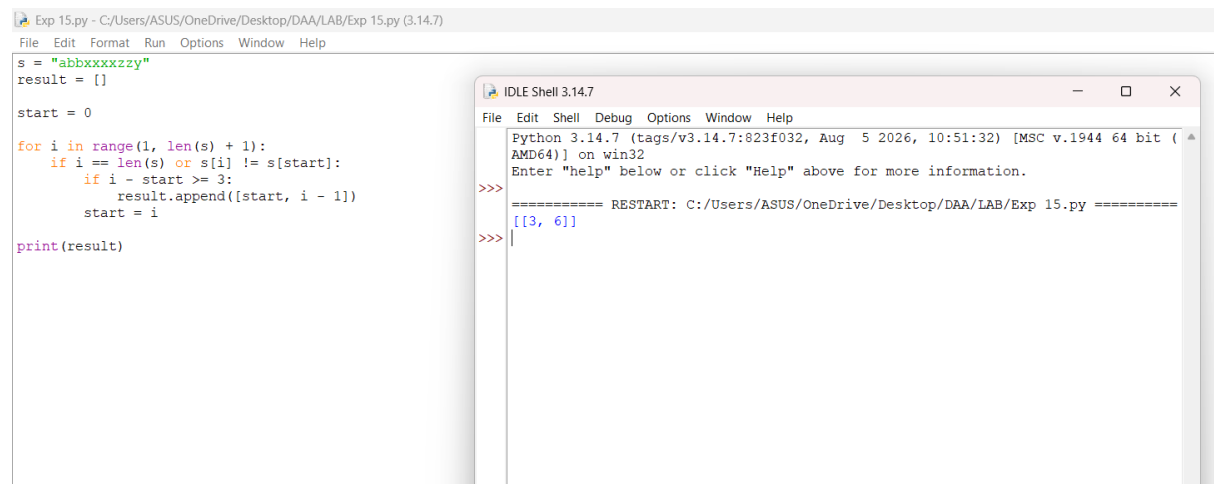
```
Exp 14.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 14.py (3.14.7)
File Edit Format Run Options Window Help

def paths(m, n):
    dp = [1] * n
    for i in range(1, m):
        for j in range(1, n):
            dp[j] = dp[j] + dp[j - 1]
    return dp[n - 1]

print(paths(7, 3))

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 14.py =====
28
>>>
```

15. Find Large Groups in a String



The screenshot shows a Python IDE window titled "Exp 15.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 15.py (3.14.7)". The code defines a function to find large groups in a string. The IDE Shell window shows the execution output, which is a list of lists: `[[3, 6]]`.

```
s = "abbxxxxzzy"
result = []

start = 0

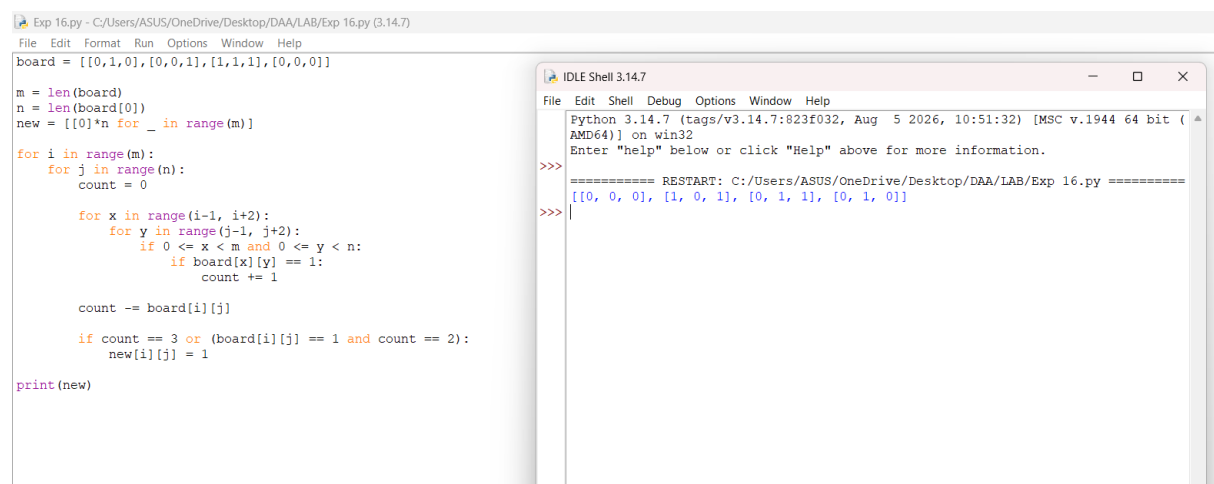
for i in range(1, len(s) + 1):
    if i == len(s) or s[i] != s[start]:
        if i - start >= 3:
            result.append([start, i - 1])
            start = i

print(result)
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 15.py =====
[[3, 6]]
>>>
```

16. Game of Life



The screenshot shows a Python IDE window titled "Exp 16.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 16.py (3.14.7)". The code implements the Game of Life algorithm. The IDE Shell window shows the execution output, which is a 2D array representing the next state of the board: `[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]`.

```
board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

m = len(board)
n = len(board[0])
new = [[0]*n for _ in range(m)]

for i in range(m):
    for j in range(n):
        count = 0

        for x in range(i-1, i+2):
            for y in range(j-1, j+2):
                if 0 <= x < m and 0 <= y < n:
                    if board[x][y] == 1:
                        count += 1

        count -= board[i][j]

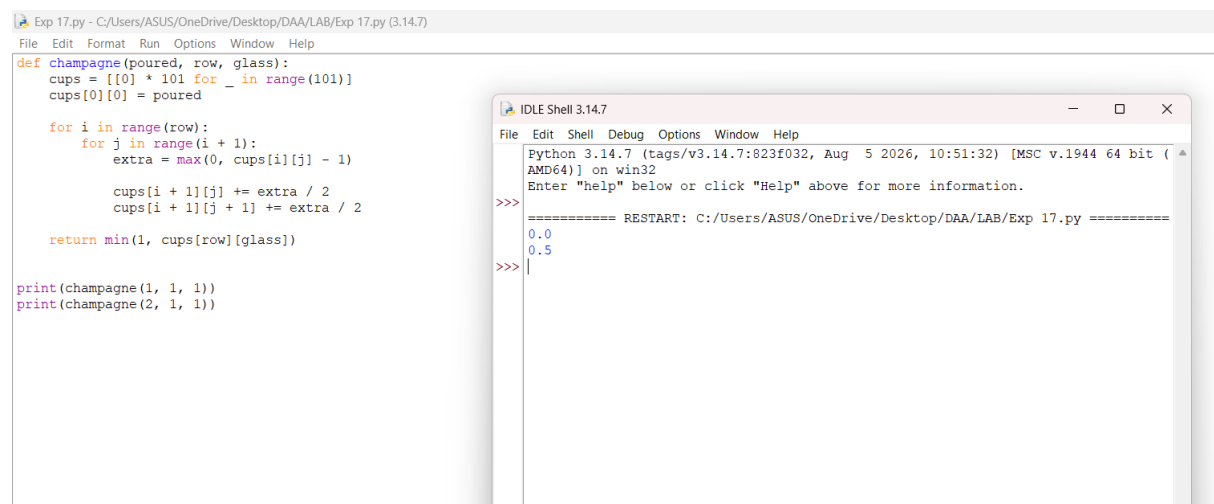
        if count == 3 or (board[i][j] == 1 and count == 2):
            new[i][j] = 1

print(new)
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 16.py =====
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
>>>
```

17. Champagne Tower



The screenshot shows a Python IDE window titled "Exp 17.py - C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 17.py (3.14.7)". The code defines a function to calculate the amount of champagne in a glass. The IDE Shell window shows the execution output, which is a list of floats: `0.0` and `0.5`.

```
def champagne(poured, row, glass):
    cups = [[0] * 101 for _ in range(101)]
    cups[0][0] = poured

    for i in range(row):
        for j in range(i + 1):
            extra = max(0, cups[i][j] - 1)

            cups[i + 1][j] += extra / 2
            cups[i + 1][j + 1] += extra / 2

    return min(1, cups[row][glass])

print(champagne(1, 1, 1))
print(champagne(2, 1, 1))
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ASUS/OneDrive/Desktop/DAA/LAB/Exp 17.py =====
0.0
0.5
>>>
```